CIS 220 Database Concepts
Lookup Tables, M:N relationships and intersection tables

Throughout this assignment I will be asking you to take screen shots.
Please place the screen shots into a single document and submit as a PDF when you have finished.

Lookup Tables:

Allowing users to enter text into an attribute that should have strict ***domain validation*** is prone to error because of the potential for misspellings. Let's look at an example using the Creature concept.

In the table below you will see a variety of misspellings of attribute values for "Creature Type"

CREATUREID	CREATURENAME	CREATURETYPE	CREATURERESIDETOWN
1	'Bannon'	'person'	'Philly'
2	'Myers'	'purson'	'Anoka'
3	'Neff'	'per son'	'Blue Earth'
4	'Neff'	'persen'	'Bemidji'
5	'Mieska'	'person'	'Duluth'
6	'Carlis'	'person'	'Edina'
7	'Kermit'	'frog'	'Greenville'
8	'Godzilla'	'monster'	'Tokyo'
9	'Thor'	'superhero'	'Asgard'
10	'The Tick'	'super hero'	'The City'
11	'American Maid'	'soup per hero'	'The City'
12	'Bat Manuel'	'superheroe'	'The City'
13	'Arthur'	'side kick'	'The City'

It should be obvious to you that these misspellings would render inaccurate query results when "creature type" is used in a WHERE clause.

The query: **SELECT * FROM creature WHERE creatureType = 'superhero';**

> Would only return **9 Thor superhero Asgard** when there are other superheroes that have their type misspelled. This type of data entry error is common.

The query: **SELECT * FROM creature WHERE creatureType = 'person';**

> Would leave out Myers and both Neffs from the results

Let's create a design that will allow us to prevent this error without having to add a bunch of validation code in the user interface. When you learn this concept you will begin to see its application everywhere.

Enter the Lookup Table: Lookup tables allow database designers to control domain selection for attribute values by placing the domain values in a table. In our current situation a lookup table could be implemented in the following fashion.

- Create a **creatureType** table. The only data that should be stored in this table are the different types of creature that exist in our system. So far we only have

    - Person
    - Frog
    - Superhero
    - Villain
    - Sidekick
    - Monster

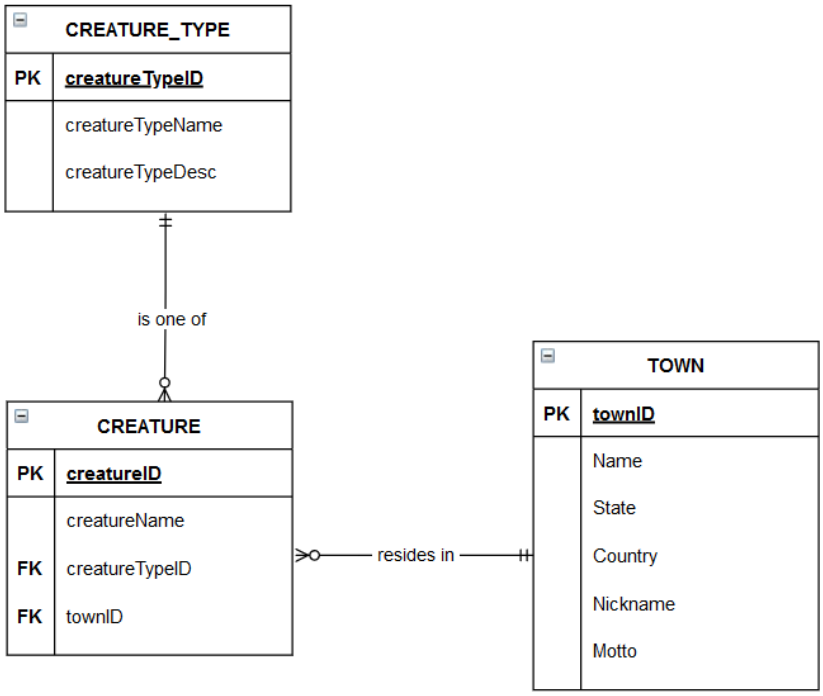
    Perhaps these are all the **creatureTypes** you will need and therefore the system should not allow for new types to be added. Perhaps your system will allow for new types to be added at will. This is a decision that your team would have to make. If there are other data that you would like to track about the type, such as a more detailed description, you could also place this in the **creatureType** table.

- **Relationship Type:** The relationship between the **CREATURE_TYPE** and **CREATURE** tables is a **1:M** and can be expressed in the following simple sentences.

    - A creature has ***one type***
    - There can be ***many creatures*** of the same type
    - Obviously this is debatable and you could also consider this a ***M:N relationship*** if your world allows creatures to be of ***many different types***. These decisions are up to you or the client. If this was the case our design would have to change to reflect this.

- To establish this 1:M relationship you take the PK from the ***one side of the relationship (CREATURE_TYPE)*** and place it as a FK in the ***many side of the relationship (CREATURE)***

    Examine the updated schema below



TASK ONE: Using the ***creature_demo*** database create the **creatureType** table by executing the following SQL **CREATE TABLE** statement

```
DROP TABLE IF EXISTS creatureType;
CREATE TABLE creatureType(
    creatureTypeID     INTEGER PRIMARY KEY AUTO_INCREMENT,
    creatureType       VARCHAR(20),
    creatureTypeDesc   TINYTEXT
) ENGINE=InnoDB;
```

Notice that the primary key for this table has been defined with the **AUTO_INCREMENT** clause. This type of attribute can be used to generate a unique identity for new rows, removing the need for us to determine what the new value will be.

Create INSERT INTO statements to add the following data into the **creatureType** table.

Creature Type	Description
Person	a member of the flawed human race
Frog	a carnivorous, short-bodied, tailless amphibian
Kaiju	giant monster that attacks major cities, the military, or other kaiju
Superhero	possesses superpowers and fits the role of the hero, using their powers to protect the public and fight crime
Villain	a cruelly malicious person who is involved in or devoted to wickedness or crime
Sidekick	a close companion or colleague who is generally regarded as subordinate to the one they accompany

Understand that you will not be inserting values for the primary key attribute so you will need to explicitly specify the columns you will be inserting into. Example:

INSERT INTO table_name (attribute1, attribute2, attribute3) VALUES (value1, value2, value3);

SUBMISSION:

1. Screenshot the **creatureType** table description
2. And the results of a **SELECT** * query.
3. Place these screenshots into your submission document.

TASK TWO: Using your ***creature_demo*** work from previous assignments, modify the CREATE TABLE statement for the **creature** table to reflect these design modifications.

Change the attribute **creatureType** to **creatureTypeID** and specify the correct datatype. Define this attribute as a foreign key that references the **creatureType** table.

Also using your work from previous assignments modify the **INSERT INTO** statements for the **creature** table to reflect the design modifications. Understand that you will need to view the table data for **creatureType** to see the values of the automatically generated primary key fields. You should expect the sequence to be 1, 2, 3, 4 . . . etc; but you should double check so that you use the correct value.

Now we have all of the various creature types stored **once** and any new creatures that are added to the database will **have** to be one of the existing **creatureTypes.**

Source these changes into the database and then write an SQL query that shows the full creature name and the full creatureType and description for each creature. Here's an example of the format I want to see.

The Tick Superhero possesses superpowers and fits the role of the hero, using their powers to protect the public and fight crime

SUBMISSION:

1. Screenshot the table descriptions for the updated **creature** table
2. the results of a **SELECT** * query and the "name, type and description" query described above
3. Be sure to include the SQL syntax for this query.
4. Place these screenshots into your submission document.

TASK THREE: Your team has decided on adding weapons and creature inventory to the system. The team has decided that a creature can be in possession of one or more weapons at any given time.

Define the **CREATE TABLE** statement to create the following "weapon" table structure. Use an auto-incremented integer primary key. If you'd like to use lookup tables for the types, please do but it is not required.

Use the following data to build your **INSERT INTO** statements

Weapon Name	Weapon Type	Damage Type	Range
club	melee	bludgeoning	2
dagger	melee	piercing	2
javelin	ranged	piercing	30
great sword	melee	bludgeoning	10
cross bow	ranged	piercing	40
long bow	ranged	piercing	60
mind control	magic	psychic	0
read thoughts	magic	psychic	0
eldritch blast	magic	bludgeoning	30

Relate the weapon table to the creature table. Things to think about:

1. What type of relationship exists between **creature** and **weapon?** Looking at the description above you are only given this simple description: "a creature can be in possession of one or more weapons at any given time". You will need to establish that the relationship is 1:M, M:N or 1:1. There is a correct answer to this. Let's chat if you are unsure.

2. Express the relationship using simple sentences. **(include these sentences as part of your submission document)**

Implement the relationship:

Appropriately modify the database structure to reflect the decisions from above. Add a new table if you must. Define CREATE TABLE statements for any new tables. Primary and foreign keys must be included and logical. Be prepared to defend your design choices here. Write some comments if you'd like. Define INSERT INTO statements that equip a collection of creatures with a collection of weapons.

Demonstrate the relationship:

Write an SQL query that shows all creatures who have weapons and the weapons they are carrying. Show the creature name and the weapon name. Do not show creatures that do not have weapons (hint: this is the nature of an ***inner join*** statement). It is perfectly fine that the creature name repeats if they have more than one weapon.

SUBMISSION:

1. Screenshot the table descriptions for any and all new tables created during this task.
2. Show the results of **SELECT** * queries for any and all new tables.
3. Also screenshot the results from the "creature, weapon" query described above and include your SQL syntax.